



LeetCode 1024: Video Stitching

1. Problem Title & Link

Title: LeetCode 1024: Video Stitching

Link: <https://leetcode.com/problems/video-stitching/>

2. Problem Statement (Short Summary)

You are given:

- A list of video clips

[start, end]

- A target duration:

time

Each clip covers a portion of the event.

You can cut clips into smaller segments if needed.

Return:

Minimum number of clips

needed to cover [0,time]

If impossible:

return -1

3. Examples (Input → Output)

Example 1

Input:

clips = [

[0,2],

[4,6],

[8,10],

[1,9],

[1,5],

[5,9]

]

time = 10

Output:

3

Explanation:

Choose:

[0,2]



[1,9]

[8,10]

Coverage:

$0 \rightarrow 10$

Example 2

Input:

clips = [[0,1],[1,2]]

time = 5

Output:

-1

Example 3

Input:

clips = [[0,4],[2,8]]

time = 5

Output:

2

4. Constraints

- $1 \leq \text{clips.length} \leq 100$
- $0 \leq \text{start} \leq \text{end} \leq 100$
- $1 \leq \text{time} \leq 100$

5. Core Concept (Pattern / Topic)

Greedy Interval Coverage

Related Topics:

- Greedy
- Intervals
- Jump Game Style

6. Thought Process (Step-by-Step Explanation)

Brute Force



Try all combinations of clips.

Check whether:

$0 \rightarrow$ time

is covered.

Complexity:

Exponential

Impossible.

Key Observation

Suppose current coverage is:

$0 \rightarrow 2$

Among all clips starting before or at:

2

we should choose the one that extends coverage the furthest.

Why?

Because:

More coverage now

can never hurt later.

This is a classic Greedy interval problem.

Similar to Jump Game II

Current reachable position:

currentEnd

Among all intervals beginning before:

currentEnd

find:

farthestEnd

When we exhaust current range:

Take one clip

and extend coverage.

7. Visual / Intuition Diagram (ASCII Diagram)

Clips:

[0, 2]

[1, 9]

[8, 10]

Coverage:



```
0----2
1-----9
      8----10
Greedy chooses:
0→2

2→9

9→10
Answer:
3
```

8. Pseudocode

```
Sort clips

currentEnd = 0
farthest = 0
clipsUsed = 0

for each clip:

    update farthest

    if current position
        exceeds currentEnd:

        clipsUsed++
        currentEnd = farthest

If coverage < time:
    return -1

return clipsUsed
```

9. Code Implementation

✓ Python

```
class Solution:

    def videoStitching(self, clips, time):

        maxReach = [0] * (time + 1)

        for start, end in clips:
```



```
        if start <= time:
            maxReach[start] = max(
                maxReach[start],
                end
            )

    clipsUsed = 0
    currentEnd = 0
    farthest = 0

    for i in range(time):

        farthest = max(
            farthest,
            maxReach[i]
        )

        if i == farthest:
            return -1

        if i == currentEnd:

            clipsUsed += 1
            currentEnd = farthest

    return clipsUsed
```

✓ Java

```
class Solution {

    public int videoStitching(
        int[][] clips,
        int time) {

        int[] maxReach =
            new int[time + 1];

        for (int[] clip : clips) {

            int start = clip[0];
            int end = clip[1];

            if (start <= time) {

                maxReach[start] =
                    Math.max(
                        maxReach[start],
                        end
                    )
            }
        }

        int clipsUsed = 0;
        int currentEnd = 0;
        int farthest = 0;

        for (int i = 0; i < time; i++) {

            farthest = Math.max(
                farthest,
                maxReach[i]
            );

            if (i == farthest)
                return -1;

            if (i == currentEnd) {

                clipsUsed++;
                currentEnd = farthest;
            }
        }

        return clipsUsed;
    }
}
```

```
        );
    }
}

int clipsUsed = 0;
int currentEnd = 0;
int farthest = 0;

for (int i = 0; i < time; i++) {

    farthest =
        Math.max(
            farthest,
            maxReach[i]
        );

    if (i == farthest)
        return -1;

    if (i == currentEnd) {

        clipsUsed++;

        currentEnd = farthest;
    }
}

return clipsUsed;
}
```

10. Time & Space Complexity

Time Complexity

$O(n + \text{time})$

where:

n = number of clips

Space Complexity

$O(\text{time})$

11. Common Mistakes / Edge Cases

Mistake 1

Sorting and selecting nearest interval.

Need:

Farthest Reach

Mistake 2

Not checking impossible coverage.

Example:

[[0,1],[3,4]]

Gap exists.

Mistake 3

Using interval overlap instead of coverage.

12. Detailed Dry Run

Input:

```
clips = [  
  [0,2],  
  [4,6],  
  [8,10],  
  [1,9],  
  [1,5],  
  [5,9]  
]
```

time = 10

Build maxReach:

Start	Max End
0	2
1	9
4	6
5	9
8	10

Traversal:

i	farthest	currentEnd	clips
0	2	0	1
1	9	2	1
2	9	9	2
8	10	9	2



9	10	10	3
---	----	----	---

Answer:

3

13. Common Use Cases

CCTV Footage Coverage

Minimum clips to cover full duration.

Network Monitoring

Cover complete time intervals.

Event Recording

Merge recordings into full event.

Resource Scheduling

Cover complete timeline with minimum tasks.

14. Common Traps

- Choosing shortest interval
- Ignoring gaps
- Not extending to farthest reach
- Treating as overlap problem

15. Builds To (Related Problems)

1. LC 45 — Jump Game II
2. LC 55 — Jump Game
3. LC 435 — Non-overlapping Intervals
4. LC 452 — Minimum Arrows to Burst Balloons
5. LC 1024 — Video Stitching

16. Alternate Approaches

Approach	Time	Space	Notes
Brute Force	Exponential	$O(n)$	Impossible
DP	$O(n \times \text{time})$	$O(\text{time})$	Good
Greedy Coverage	$O(n + \text{time})$	$O(\text{time})$	Optimal



17. Why This Solution Works

At every point, we greedily choose the clip that extends the current coverage the farthest. Any other choice would provide less coverage and can never lead to a better solution.

18. Variations / Follow-Up Questions

Follow-Up 1

Minimum intervals to cover a range.

Same Greedy Interval pattern.

Follow-Up 2

What if clips have costs?

Use Dynamic Programming.

Follow-Up 3

What if intervals are added dynamically?

Use Interval Trees or Segment Trees.